



Universidad de las Fuerzas Armadas ESPE

Modelos Discretos

INTEGRANTES

Julio Andrade
Edwin Guachi

DOCENTE

Washington Eduardo Loza Herrera

[Departamento de Ciencias de la Computación]

LÓGICA DEL SISTEMA: ESTRUCTURAS Y MAPEO

main.cpp

```
struct EstadoLinea {
    int meta; // 0:Baja, 1:Media, 2:Alta
    int falla; // 0:No, 1:Si
    int validado; // 0:No, 1:Si
};

void imprimirMetaTeorico(int meta) {
    if (meta == 0) cout << "Baja";
    else if (meta == 1) cout << "Media";
    else cout << "Alta";
}

void imprimirSiNoTeorico(int valor) {
    if (valor == 1) cout << "Si";
    else cout << "No";
}

int contarEstadosPosibles(int nMeta, int nFalla, int nValidado) {
    return nMeta * nFalla * nValidado;
}
```



1. Estructura de Datos

Definimos **EstadoLinea** con variables elementales discretas que representan la meta de producción, presencia de fallas y la validación final.



2. Mapeo Cualitativo

La función **textoMeta** asocia enteros discretos (0, 1, 2) a las categorías legibles de meta: **Baja**, **Media** y **Alta**.



3. Lógica Si/No Normalizada

Normalizamos los estados de falla o supervisión usando un operador ternario para retornar **"Si"** en caso de 1 o **"No"** en cualquier otro caso.



4. Cálculo de Espacio Muestral

Aplicación de lógica matemática combinatoria para obtener de forma directa las dimensiones del espacio muestral completo.

PRINCIPIO MULTIPLICATIVO

"Determinar los posibles estados de producción considerando cumplimiento de metas, fallos mecánicos y validación del supervisor de la línea." . Posteriormente, se deben usar combinaciones para analizar escenarios donde el orden de los eventos no altera el estado final de la línea."

Total de formas = $n_1 \times n_2 \times n_3 \times \dots$

$$3 \times 2 \times 2 = 12$$

Desglose de estados:

- Meta: Baja, Media, Alta (3)
- Fallo: Si, No (2)
- Validado: Si, No (2)

Este enfoque metodológico se implementa ante la exigencia técnica de realizar un inventario exhaustivo y riguroso de la totalidad de estados de producción o componentes elementales que constituyen el conjunto del sistema.

** Usamos este principio para obtener el espacio muestral completo y enumerar cada elemento del conjunto de estados.*

ENUMERACIÓN DEL ESPACIO MUESTRAL

listar.cpp

```
void listarEstadosTeoricos() {  
    int contador = 1;  
    for (int meta = 0; meta < 3; meta++) {  
        for (int falla = 0; falla < 2; falla++) {  
            for (int validado = 0; validado < 2; validado++) {  
                cout << " " << contador << ". Meta=";  
                imprimirMetaTeorico(meta);  
                cout << " | Falla=";  
                imprimirSiNoTeorico(falla);  
                cout << " | Validado=";  
                imprimirSiNoTeorico(validado);  
                cout << "\n";  
                contador++;  
            }  
        }  
    }  
}
```



Triple Iteración Anidada

Implementación lógica del producto cartesiano mediante bucles **for** para recorrer el espacio muestral.



Frecuencia de Validación (1:1)

El bucle interno más profundo alterna el estado de validación (Si/No) en cada iteración individual.



Frecuencia de Fallo (2:1)

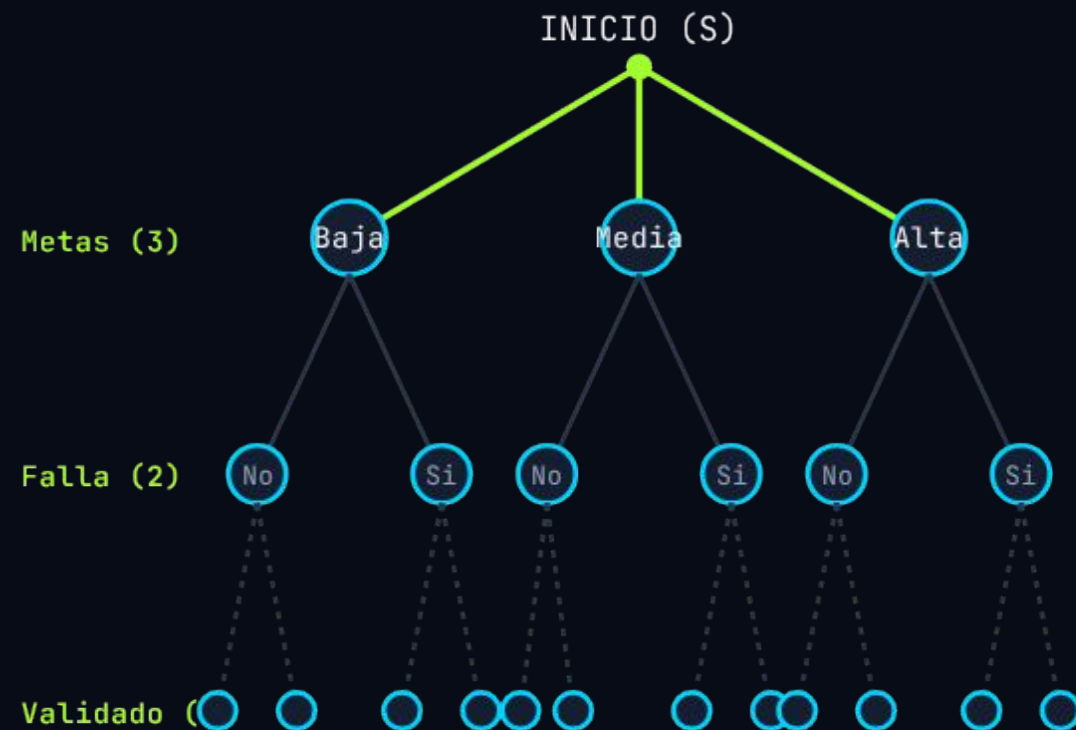
El estado mecánico (Falla) cambia cada **2 iteraciones** completas del bucle de validación.



Frecuencia de Meta (4:1)

El nivel de meta (Bajo, Medio, Alto) se mantiene constante durante cada ciclo de **4 iteraciones**.

VISUALIZACIÓN DE ITERACIONES



#	META	FALLA	VALIDADO
1	Baja	No	No
2	Baja	No	Si
3	Baja	Si	No
4	Baja	Si	Si
5	Media	No	No
6	Media	No	Si
7	Media	Si	No
8	Media	Si	Si
9	Alta	No	No
10	Alta	No	Si
11	Alta	Si	No
12	Alta	Si	Si

ANÁLISIS COMBINATORIO Y RECURSIÓN

```
long long factorial(int n) {  
    if (n == 0) {  
        return 1; // 0! = 1 por definición  
    }  
    return n * factorial(n - 1);  
}  
  
long long combinaciones(int n, int k) {  
    if (k < 0 || k > n) return 0;  
    return factorial(n) / (factorial(k) * factorial(n - k));  
}
```

Cálculo de Factorial

Implementación recursiva para obtener el factorial. Caso base: si n es 0, retorna 1 por definición (0! = 1).

Recursión Algorítmica

La función se llama recursivamente disminuyendo n hasta el caso base, multiplicándose por sí mismo en el retorno.

Combinación Simple

Aplicación de la fórmula para subconjuntos donde el orden no es relevante.

$$C_{n,k} = \frac{n!}{k! (n-k)!}$$

Análisis de Tiempo de Ejecución en Segundos $T(n)$

ANÁLISIS DE TIEMPO DE EJECUCIÓN EN SEGUNDOS $T(N)$

Función: listarEstadosTeoricos()

```
void listarEstadosTeoricos() {  
    int contador = 1; 1 OE  
    for (int meta = 0; meta < 3; meta++) {  
        1 OE  
        for (int falla = 0; falla < 2; falla++) {  
            1 OE  
            for (int validado = 0; validado < 2; validado++) {  
                2 OE  
                cout << ".." << contador << ". Meta="; 1 OE  
                imprimirMetaTeorico(meta); 5 OE ← (Función previa)  
                cout << " | Falla="; 1 OE  
                imprimirSiNoTeorico(falla); 3 OE ← (Función previa)  
                cout << " | Validado="; 1 OE  
                imprimirSiNoTeorico(validado); 3 OE ← (Función previa)  
                cout << "\n"; 1 OE  
                contador++; 2 OE  
            }  
        }  
    }  
}
```

LEYENDA DE OE

- 1 OE Inicialización / Comparación
- 2 OE Incremento (i++)
- 1 OE Operaciones de salida (cout)
- 5 OE Llamada a imprimirMetaTeorico()
- 3 OE Llamada a imprimirSiNoTeorico()

FUNCIONES PREVIAS UTILIZADAS

- imprimirMetaTeorico(meta)
→ 5 OE
- imprimirSiNoTeorico(...)
→ 3 OE

RESULTADO FINAL:

$T(N) = 777 \text{ OE} \rightarrow O(1) \rightarrow \text{COMPLEJIDAD CONSTANTE}$

CÁLCULO DEL TIEMPO $T(N)$

$$10E + 10E + 10E + \sum_{i=0}^{3-1} \left(10E + 10E + \sum_{j=0}^{2-1} \left(10E + 10E + \sum_{k=0}^{2-1} (10E + 10E + 10E + 10E + 50E + 10E + 10E + 10E + 30E + 10E + 10E + 10E + 30E + 10E + 20E) + 10E + 20E \right) + 10E + 10E \right)$$

$$300E + (3) \{ 200E + (2) [200E + 5400E + 300E] + 300E \}$$

$$300E + (3) \{ 200E + (2) [10400E] + 300E \}$$

$$300E + (3) \{ 200E + 2080E + 300E \}$$

$$300E + (3) \{ 2580E \}$$

$$300E + 7740E$$

777 OE

NOTA:

- Hay 3 ciclos anidados con límites constantes (3, 2 y 2).
- El número total de estados siempre es $3 \times 2 \times 2 = 12$.
- Por lo tanto, el tiempo de ejecución es constante: $O(1)$.

```
void imprimirMetaTeorico(int meta) {  
    1 OE  
    if (meta == 0) cout << "Baja"; 1 OE  
    1 OE  
    else if (meta == 1) cout << "Media";  
    1 OE  
    else cout << "Alta";  
}
```

$T(n) = 1 \text{ OE} + 1 \text{ OE} + 1 \text{ OE} + 1 \text{ OE} + 1 \text{ OE}$
 $T(n) = 5 \text{ OE}$
Orden constante

Función: imprimirSiNoTeorico(int valor)

```
void imprimirSiNoTeorico(int valor) {  
    1 OE  
    if (valor == 1) cout << "Si"; 1 OE  
    1 OE  
    else cout << "No";  
}
```

$T(n) = 1 \text{ OE} + 1 \text{ OE} + 1 \text{ OE}$

$T(n) = 3 \text{ OE}$

Orden constante

ECUACIÓN DE RECURRENCIA DEL SISTEMA



```
long long costoRecurrencia(int n) {  
    if (n == 0) return 0;  
    return costoRecurrencia(n - 1) + 777;  
}
```

Ecuación de Recurrencia

Esta función representa matemáticamente la recurrencia de nuestro código: $T(n) = T(n-1) + O(1)$, donde cada llamada decrementa el tamaño en 1 y suma el costo constante de las 777 OE.

Complejidad Lineal $O(n)$

Al resolverse la recurrencia de manera sucesiva para un tamaño de entrada n , el número total de operaciones escala proporcionalmente, resultando en un comportamiento de **crecimiento lineal**.

VERIFICACIÓN DE RELACIONES Y ESTRUCTURA DISCRETA

```
bool esMenorOIgualTeorico(const EstadoTeorico& a, const EstadoTeorico& b) {
    return (b.meta ≥ a.meta) && (b.falla ≤ a.falla) && (b.validado ≥ a.validado);
}

bool sonIgualesTeorico(const EstadoTeorico& a, const EstadoTeorico& b) {
    return (a.meta == b.meta) && (a.falla == b.falla) && (a.validado == b.validado);
}

EstadoTeorico obtenerEstadoPorIndice(int indice) {
    int meta = indice / 4;
    int resto = indice % 4;
    int falla = resto / 2;
    int validado = resto % 2;
    EstadoTeorico e = { meta, falla, validado };
    return e;
}

bool existeIntermedio(const EstadoTeorico& a, const EstadoTeorico& b, int
totalEstados) {
    for (int i = 0; i < totalEstados; i++) {
        EstadoTeorico c = obtenerEstadoPorIndice(i);
        bool esDistintoDeAyB = !sonIgualesTeorico(c, a) && !sonIgualesTeorico(c, b);
        if (esDistintoDeAyB && esMenorOIgualTeorico(a, c) && esMenorOIgualTeorico(c,
b)) {
            return true;
        }
    }
    return false;
}
```

Mapeo Discreto por Índices

La función `obtenerEstadoPorIndice` decodifica una posición lineal mediante aritmética modular para extraer sus 3 variables algebraicas elementales con costo $O(1)$.

Orden Parcial y Comparación

Las funciones lógicas evalúan las condiciones de igualdad y dominancia (`esMenorOIgualTeorico`) para estructurar el reticulado de estados discreto de manera booleana.

Complejidad del Bucle i: $O(\text{totalEstados})$

La función `existeIntermedio` busca la existencia de un estado `c` acotado estrictamente entre `a` y `b`. Al iterar linealmente sobre el espacio muestral, su complejidad temporal depende de $O(N)$ instrucciones elementales.

CONSTRUCCIÓN DEL DIAGRAMA DE HASSE

```
void construirDiagramaHasse(int totalEstados) {
    encabezado("DIAGRAMA DE HASSE (relaciones de cobertura)");
    cout << " Lectura: 'A → B' significa que B es el siguiente nivel\n";
    cout << " de eficiencia inmediatamente superior a A.\n\n";

    for (int i = 0; i < totalEstados; i++) {
        EstadoTeorico a = obtenerEstadoPorIndice(i);
        for (int j = 0; j < totalEstados; j++) {
            if (i == j) continue;
            EstadoTeorico b = obtenerEstadoPorIndice(j);

            bool esRelacionValida = esMenorOIgualTeorico(a, b) && !sonIgualesTeorico(a,
b);
            if (esRelacionValida && !existeIntermedio(a, b, totalEstados)) {
                // Bloque de impresión del enlace transicional
                cout << " [Meta="; imprimirMetaTeorico(a.meta);
                cout << ",Falla="; imprimirSiNoTeorico(a.falla);
                cout << ",Val="; imprimirSiNoTeorico(a.validado);
                cout << "] → [Meta="; imprimirMetaTeorico(b.meta);
                ...
            }
        }
    }
}
```

Filtrado de Cobertura Matemática

→ Un Diagrama de Hasse requiere omitir aristas redundantes por transitividad. El código asegura esto validando que $a < b$ y verificando que `!existeIntermedio(a, b)`.

Complejidad Temporal Cuadrática: $O(N^2)$

→ El algoritmo cuenta con dos bucles anidados que recorren `totalEstados (N)`. Dado que adentro invoca a `existeIntermedio` (que es $O(N)$), la complejidad total en el peor de los casos es $O(N^2)$.